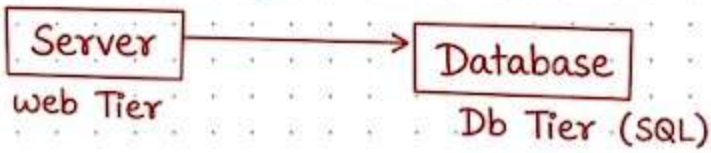
Lecture 15 : Internal Architecture of SQL | B/B+ Trees | Indexing

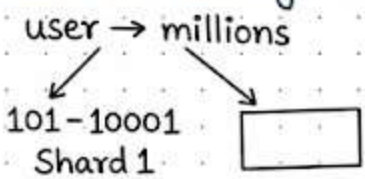


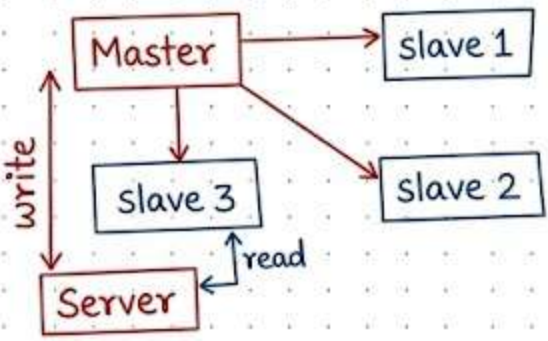
Master - slave Architecture

User (PK)

UserId	name	email
101	Aditya	xyz
102	Rohit	ert
103	Mukesh	wer

Horizontal scaling → Sharding





Views → Virtual Tables

SQL DB (PK)

user

UserId	name	email
101	Aditya	xyz
102	Rohit	ert
103	Mukesh	wer

Relational DB → Rows and columns

(Logical Arrangment)

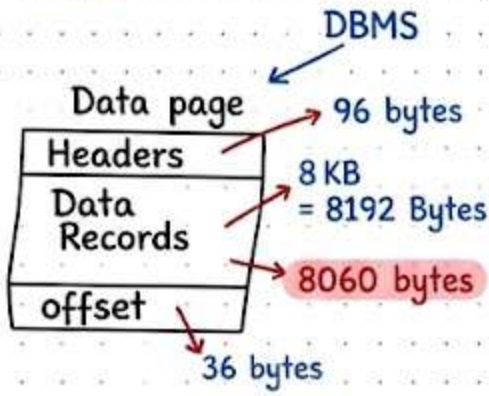
- Stored not in volatile memory
- Stored in permanent memory (Disk)



↳ Data Pages

UserID	name	email
101	Aditya	xyz
102	Rohit	ext
103	Mukesh	wer

1 Million Rows



1 row = 32 bytes

1 Data Records = ~251 rows

Total page = 3984 pages

↳ Headers :
  ↳ stores page Number
  ↳ stores checksum

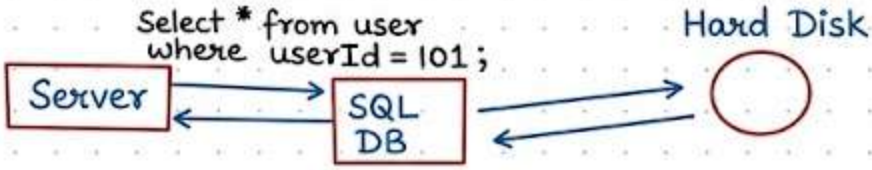
↳ Offset :
  ↳ Array of indexed rows

* Datapages are managed by DBMS
* But practically , All data pages will be stored in memory

Data Blocks: It is the smallest possible value we can expect from an IO operation.

1 Data Block → 8KB - 64 KB

If 1 data block = 8KB
1 data block = 1 Data page



Data Block 1 - page 1, page 2
Data Block 2 - page 3, page 4
Data Block 3 -

O(N) time complexity → 10 million

↳ Data Blocks are managed by Hard Disk
↳ DBMS manages Data pages.

DBMS has a mapping table
→ Data page 1 → Data block 1
Data page 2 → Data block 1
Data page 3 → Data block 2

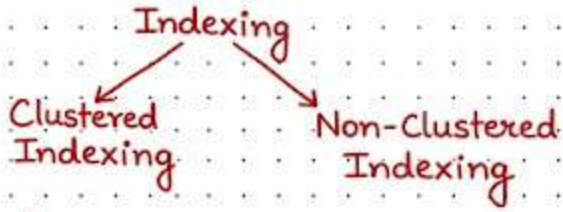
Data page ⟶ Data block

Indexing:

- Used to increase the performance of our DB queries.
  → Making Queries run faster.
- Without indexing → O(N) → Reduce my search result

DSA → Binary search / BST
B+ Tree are used for DB indexing.

O(N) ⟶ O(log N)

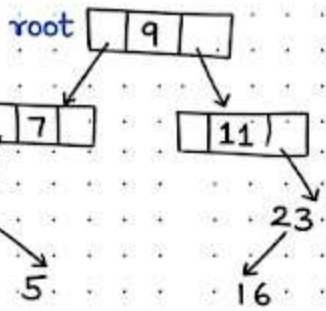


↳ B Trees / B+ Trees

→ B Trees

9, 7, 11, 3, 23, 5, 16

in BST:

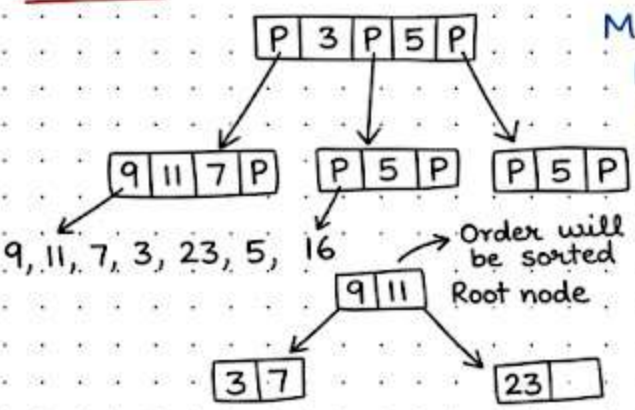



```
BST
 { int data;
   BST left;
   BST right;
 }
```

M-way
M = 2
keys = 1 (M-1)

BST → Binary Search Tree (2)
TST → Ternary Search Tree (3)
M-Tree → M children (M)

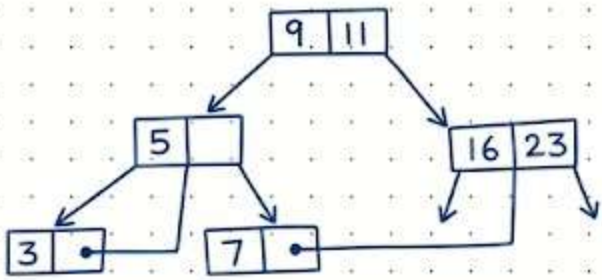
B Tree:



M-way tree
M = 3
K = M-1 (3-1) = 2

```
{ int data1;
  int data2;
  Btree P1;
  Btree P2;
  Btree P3;
}
```

↓



B+ Trees :

↓ ↳ leaf nodes connected

→ intermediate nodes also store parent value

Indexing
↓
B+ Tree

- → Better because all the data will be present in sorted order in leaf nodes
- deletion slows indexing in B Tree

PK user

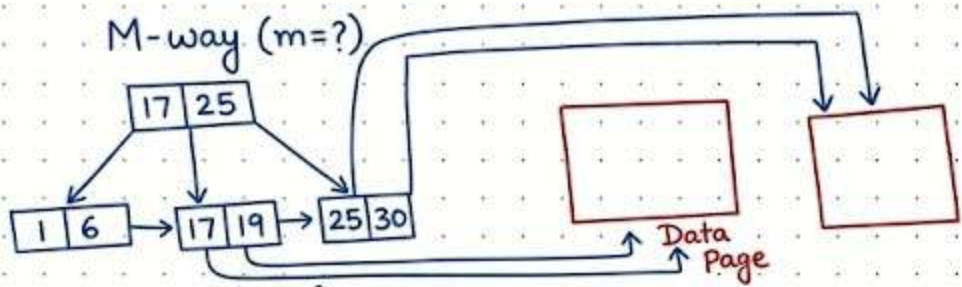
userId	name	email
101	Aditya	xyz
103	Rohit	ert
102	Mukesh	wer

1 Million records

→

101, Aditya, xyz
103, Rohit, ert

Data page



SELECT * from user
where userId = 19 ;

DBMS
data page → Data block



Datapage



Offset : Arrays of indexed row in which offsets stores the data index in sorted way

Indexing

- Clustered
- Non-Clustered

Leaf nodes of B+ Trees store complete row information

which we studied, it just tell page no. we have to see offset

- Leaf nodes of BT tree store only id information (page no.)

↳ They can be only 1 clustered ~~index~~ index per table

↳ there can be multiple non clustered index per table.

What is the issue to index each column?

↳ It will require a lot of space.